SIMATS Engineering • CSE • CSA0802 — Python Programming • 16-08-2026 • Category: Class Work (11.8.26) • Student Submissions Report

Student 5 **Deveswar A** 192512152RC 10 / 10 submitted

Q1. Selection Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
n = list(map(int,input().split()))
print (list(sorted(n)))
```

INPUT

```
9 5 1 7
```

OUTPUT

```
[1, 5, 7, 9]
```

Q2. Insertion Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
n = list(map(int, input().split()))

for i in range(1, len(n)):
    key = n[i]
    j = i - 1

    while j ≥ 0 and n[j] > key:
        n[j + 1] = n[j]
        j -= 1

    n[j + 1] = key

print(n)
```

INPUT

```
8 5 7 1 9 3
```

OUTPUT

```
[1, 3, 5, 7, 8, 9]
```

Q3. Merge Sort

Score: 10.00 **TC: 3/3 passed**

CODE

```
def merge_sort(a):
    if len(a) ≤ 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result += left[i:]
    result += right[j:]
    return result
n = list(map(int, input().split()))
print(merge_sort(n))
```

INPUT

```
38 27 43 3 9 82 10
```

OUTPUT

```
[3, 9, 10, 27, 38, 43, 82]
```

Q4. Diagonal Matrix

Score: 10.00 **TC: 3/3 passed**

CODE

```
import json
n = json.loads(input())
diagonal = True
for i in range(len(n)):
   for j in range(len(n)):
      if i ≠ j and n[i][j] ≠ 0:
         diagonal = False
         break
if diagonal:
    print("Diagonal Matrix")
else:
    print("Not Diagonal Matrix")
```

INPUT

```
[[1,2],[0,3]]
```

OUTPUT

```
Not Diagonal Matrix
```

Q5. Row and Column Sums

Score: 6.67 **TC: 2/3 passed**

CODE

```
import json
n = json.loads(input())
row = []
col = []
for i in range(len(n)):
    row.append(sum(n[i]))
for j in range(len(n[0])):
    s = 0
    for i in range(len(n)):
        s += n[i][j]
    col.append(s)
print("Row=", row)
print("Col=", col)
```

INPUT

```
[[5,6],[7,8]]
```

OUTPUT

```
Row= [11, 15]
Col= [12, 14]
```

Q6. Maximum Subarray Sum

Score: 6.67 **TC: 2/3 passed**

CODE

```
n = list(map(int, input().split()))
max_sum = n[0]
current = n[0]
for i in range(1, len(n)):
    current = max(n[i], current + n[i])
    max_sum = max(max_sum, current)
print(max_sum)
```

INPUT

```
-1 -2 -3
```

OUTPUT

```
-1
```

Q7. Common Elements Among Three Lists

Score: 0.00 **TC: 0/3 passed**

CODE

```
a = list(map(int, input().split()))
b = list(map(int, input().split()))
c = list(map(int, input().split()))
result = []
for x in a:
    if x in b and x in c and x not in result:
        result.append(x)
print(result)
```

INPUT

```
1 2
3 4
5 6
```

OUTPUT

```
[]
```

Q8. Matrix Multiplication

Score: 0.00 **TC: 0/3 passed**

CODE

```
import json
A = json.loads(input("A="))
B = json.loads(input("B="))
result = []
for i in range(len(A)):
    row = []
    for j in range(len(B[0])):
        s = 0
        for k in range(len(B)):
            s += A[i][k] * B[k][j]
        row.append(s)
    result.append(row)
print(result)
```

INPUT

```
[[1,2],[3,4]]
[[5,6],[7,8]]
```

OUTPUT

```
A=B=[[19, 22], [43, 50]]
```

Q9. Sort Rotate Min Max

Score: 0.00 **TC: 0/3 passed**

CODE

```
lst = [3, 1, 2]
k = 1
lst.sort()
print("Sorted=", lst)
k = k % len(lst)
rotated = lst[-k:] + lst[:-k]
print("Rotated=", rotated)
print("Min=", min(lst))
print("Max=", max(lst))
```

OUTPUT

```
Sorted= [1, 2, 3]
Rotated= [3, 1, 2]
Min= 1
Max= 3
```

Q10. Sort Matrix Search

Score: 0.00 **TC: 0/3 passed**

CODE

```
matrix = [[9, 3], [7, 1]]
find = 7

elements = []
for row in matrix:
    elements.extend(row)

elements.sort()

rows = len(matrix)
cols = len(matrix[0])

sorted_matrix = []
for i in range(rows):
    sorted_matrix.append(elements[i * cols:(i + 1) * cols])

print("Sorted =", sorted_matrix)

found = False
for i in range(rows):
    for j in range(cols):
        if sorted_matrix[i][j] == find:
            print("Position =", (i, j))
            found = True

if not found:
    print("Not Found")
```

OUTPUT

```
Sorted = [[1, 3], [7, 9]]
Position = (1, 0)
```